

UNIVERSIDAD NACIONAL TORIBIO RODRÍGUEZ DE MENDOZA
FACULTAD DE INGENIERÍA DE SISTEMAS Y MECÁNICA ELÉCTRICA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



INTEGRANTES:

ALTAMIRANO HORNA JARED AMIR

CAMPOS MARCO CRISTIAN LENIN

JEFFERSON LEÓN CHUPILLON

DOCENTE:

Dr. Ing. Roberto Pérez Astonitas

Ingeniería del Software I

25 de abril 2025

Bagua – Amazonas – Perú

Tipos de Metodologías de Desarrollo de Software

Introducción

Imagina que vas a construir una casa o armar un robot. No empiezas a poner ladrillos o tornillos al azar: necesitas un plan, seguir ciertos pasos y organizar a tu equipo. Lo mismo sucede cuando se crea un programa de computadora, una aplicación para el celular o una página web. Para hacer todo esto correctamente, los desarrolladores usan algo llamado metodologías de desarrollo de software.

Estas metodologías son formas de trabajar, como recetas que indican qué hacer primero, qué hacer después y cómo hacerlo. Existen tres tipos principales: metodologías ágiles, tradicionales e híbridas. A continuación, te las explicamos paso a paso.

1. Metodologías Ágiles

¿Qué son? (Descripción)

Las metodologías ágiles son flexibles, rápidas y se enfocan en el trabajo en equipo y en mejorar constantemente. En vez de hacer todo el proyecto de una sola vez, se divide en partes pequeñas. Cada parte se desarrolla, se prueba, se corrige y luego se pasa a la siguiente.

Es como si fueras armando un videojuego por niveles: primero haces el nivel 1, lo pruebas, lo mejoras si hace falta, y luego haces el nivel 2.

Estas metodologías permiten adaptarse si el cliente cambia de idea o si surgen nuevas necesidades durante el desarrollo.

Fases de una metodología ágil

Recolección de requerimientos:

Se conversa con el cliente o usuario para entender qué necesita el programa. Por ejemplo, si alguien quiere una app para vender postres, hay que saber qué funciones debe tener.

Planificación en pequeños ciclos (sprints):

El trabajo se organiza en etapas cortas (de una a cuatro semanas), llamadas *sprints*. Cada sprint se enfoca en completar una parte específica del proyecto.

Desarrollo:

Se programa la parte del software correspondiente a ese sprint, trabajando en equipo y revisando constantemente.

Pruebas:

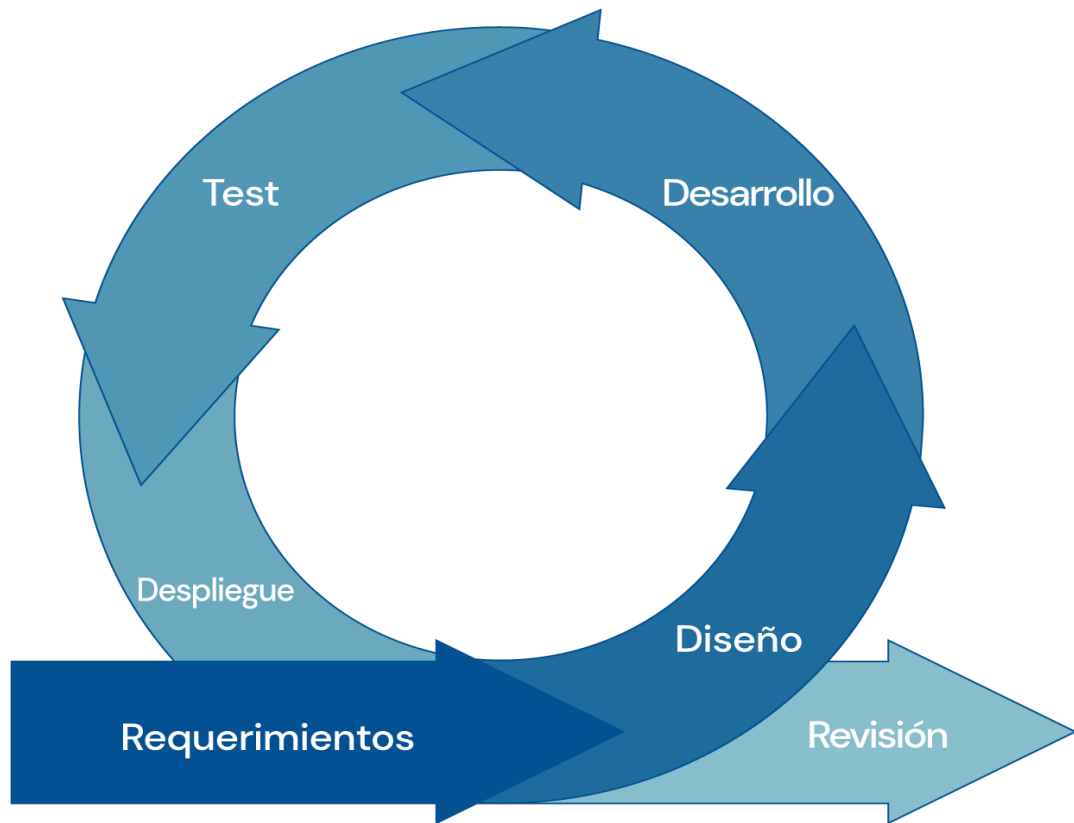
Se revisa si lo que se ha creado funciona bien. Se corrigen errores rápidamente.

Retroalimentación del cliente:

El cliente o usuario prueba lo que se ha hecho y dice si está conforme o si necesita ajustes.

Mejoras y repetición del proceso:

Se corrigen los detalles y se pasa al siguiente sprint con otra parte del proyecto.



Importancia

Este método permite trabajar de forma más cercana al cliente y hace que el software mejore continuamente. Es muy útil si no se sabe exactamente cómo debe ser el producto final desde el principio.

Utilidad

Desarrollo de apps móviles o videojuegos que cambian con el tiempo.

Proyectos donde el cliente tiene nuevas ideas durante el trabajo.

Equipos creativos que trabajan en entornos cambiantes o innovadores.

2. Métodos Tradicionales

¿Qué son? (Descripción)

Los métodos tradicionales siguen un orden fijo, como una línea recta. Todo se planifica desde el inicio, y no se pasa a la siguiente etapa hasta que la anterior esté completamente terminada. A este enfoque también se le llama “modelo en cascada” porque las fases caen una tras otra como una cascada.

Es como construir un edificio: primero haces los planos, luego levantas las paredes, después el techo, y así hasta terminar. Si te equivocas en el plano, arreglarlo cuando ya estás en la etapa del techo es muy difícil.

Fases de un método tradicional

Análisis de requisitos:

Se define con todo detalle lo que el software debe hacer. Por ejemplo, qué botones tendrá, qué funciones realizará, cómo se verá.

Diseño:

Se crea un esquema de cómo funcionará internamente el programa. Se deciden cosas técnicas y visuales.

Implementación (programación):

Los programadores escriben el código según lo planificado.

Pruebas:

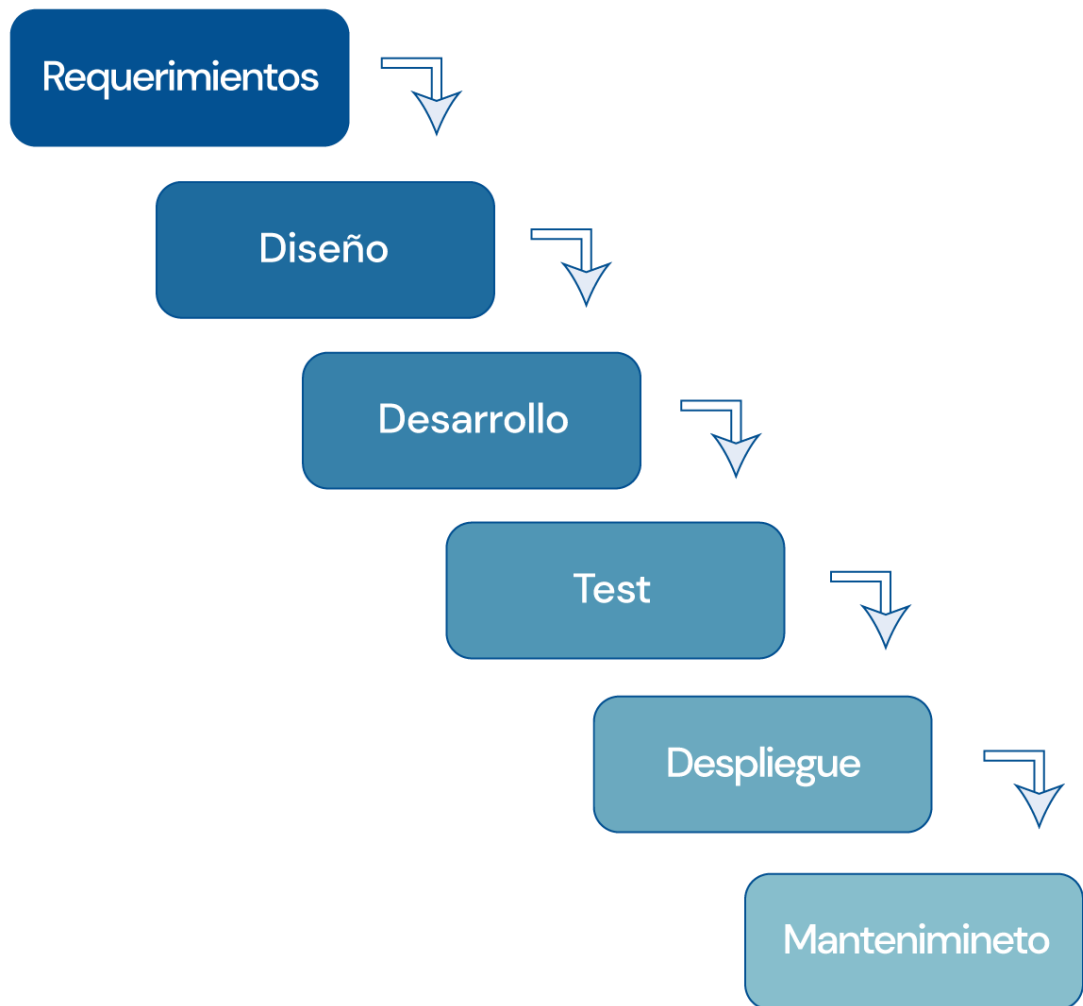
Se revisa que el software funcione correctamente, buscando errores o fallos.

Entrega al cliente:

El producto terminado se entrega al cliente. No se espera que haya cambios.

Mantenimiento:

Si luego aparece un error o el cliente quiere hacer un pequeño cambio, se hacen arreglos.



Importancia

Este tipo de metodología es importante cuando todo debe estar bien planificado desde el principio. Es útil si el cliente ya sabe exactamente qué quiere y no va a cambiar de opinión.

Utilidad

Programas para bancos, hospitales o empresas donde todo debe estar controlado.

Proyectos grandes y largos donde no se pueden hacer muchos cambios.

Situaciones donde se necesita cumplir reglas muy estrictas o certificaciones.

3. Metodologías Híbridas

¿Qué son? (Descripción)

Las metodologías híbridas combinan lo mejor de las metodologías ágiles y tradicionales. Son como una mezcla: permiten tener una estructura organizada, pero también dejan espacio para adaptarse y mejorar durante el desarrollo.

Es como tener un plan para construir una casa, pero permitir que el dueño diga “quiero cambiar el color del cuarto” a mitad del proceso.

Fases de una metodología híbrida

Planificación general:

Se establece una visión global del proyecto, con metas y fechas importantes.

División en fases ágiles:

El trabajo se separa en partes que se pueden hacer al estilo ágil (por ejemplo, por módulos o funciones).

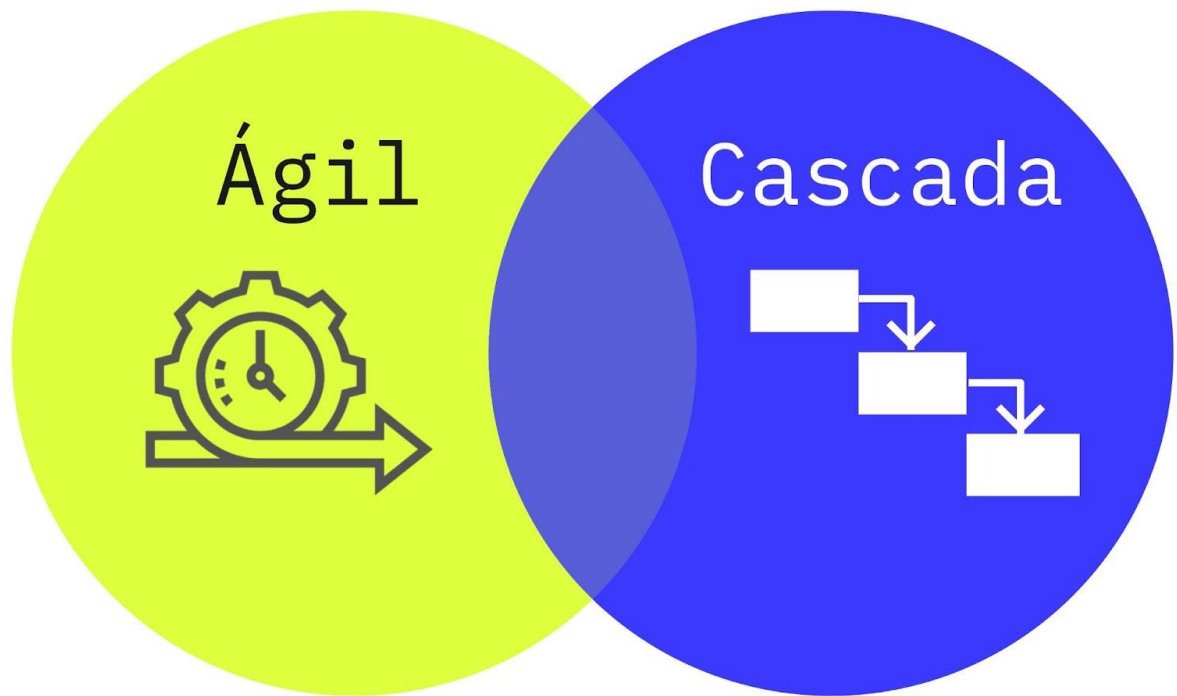
Desarrollo con flexibilidad:

Se desarrollan y prueban las partes del software en pequeños ciclos, permitiendo cambios si es necesario.

Control y seguimiento:

Se supervisa que el proyecto avance según lo planeado, con reuniones y revisiones periódicas.

Híbrido

**Importancia**

Este método es ideal cuando se necesita tener cierta organización formal, pero también se desea responder rápidamente a los cambios.

Utilidad

Proyectos en empresas grandes que trabajan con clientes exigentes.

Cuando hay que cumplir reglas, pero también responder al mercado.

Proyectos donde participan muchas personas y equipos distintos.

Metodología de Desarrollo de Software - XP (Extreme Programming/ *Programación Extrema*)

¿Qué es XP? (Descripción)

XP o Extreme Programming es una metodología ágil que se enfoca en hacer el mejor software posible en el menor tiempo, a través de la colaboración continua, mejoras constantes y una forma de trabajar sencilla, pero efectiva.

Se llama "extrema" porque lleva al extremo las buenas prácticas de programación. Por ejemplo, en lugar de revisar el código de vez en cuando, se revisa todo el tiempo. En lugar de probar el programa al final, se prueba desde el principio y a cada rato.

XP fue diseñada para equipos pequeños que trabajan en proyectos que pueden cambiar mucho a lo largo del tiempo, y donde es muy importante estar siempre en contacto con el cliente.

Fases de XP (Proceso de trabajo)

Aunque XP no tiene fases tan rígidas como las metodologías tradicionales, sí sigue un proceso muy claro que incluye varias prácticas importantes:

Planificación del lanzamiento

Aquí el equipo converso con el cliente para entender qué quiere y necesita. Se hacen unas listas llamadas "historias de usuario", que explican de forma sencilla lo que el programa debe hacer, como si fueran mini cuentos.

Ejemplo: *"Como usuario, quiero poder guardar mi progreso en el juego para continuar después."*

El equipo estima cuánto tiempo tomará hacer cada historia, y se arma un plan general del proyecto.

Iteraciones cortas (Ciclos de trabajo)

El trabajo se divide en pequeñas partes (normalmente de 1 a 2 semanas), llamadas iteraciones. En cada una se eligen algunas historias y se trabaja en ellas hasta completarlas.

Esto permite tener resultados visibles rápidamente y hacer cambios si es necesario.

Diseño simple

El equipo hace diseños fáciles de entender. La idea es no complicarse demasiado desde el inicio, sino hacer algo que funcione y que pueda mejorarse poco a poco.

Programación en pareja

Una característica muy especial de XP es que dos personas trabajan en una misma computadora al mismo tiempo. Mientras una escribe el código, la otra revisa y da ideas. Esto se llama *programación en pareja*, y mejora la calidad del software.

Pruebas constantes

En XP, se hacen pruebas desde el principio. Antes de escribir el código, los programadores crean pruebas que el programa debe pasar. Esto se llama *desarrollo guiado por*

pruebas (o TDD por sus siglas en inglés). Así se aseguran de que todo funcione bien desde el primer momento.

Integración continua

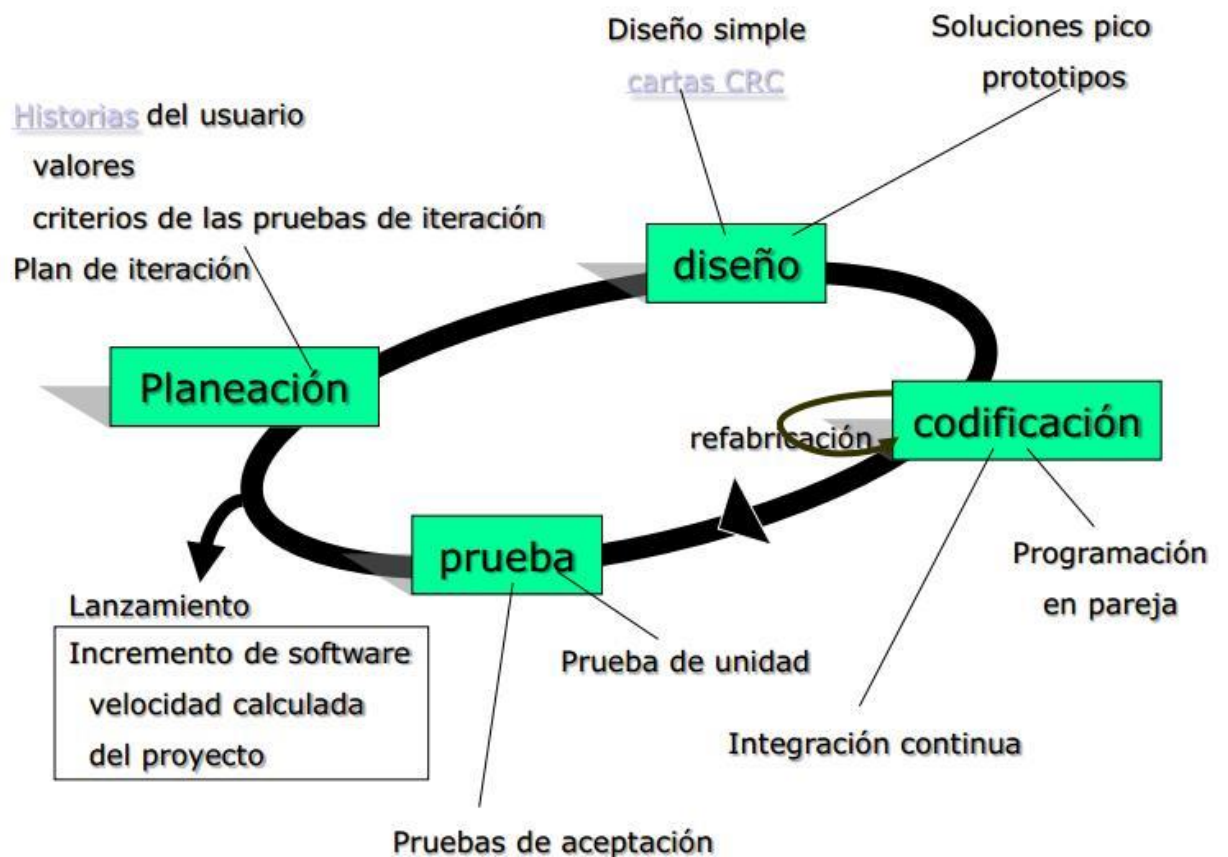
Cada vez que se termina una parte del programa, se une con el resto del proyecto. Esto se hace muchas veces al día para evitar errores grandes y mantener todo funcionando correctamente.

Retroalimentación rápida

Después de cada iteración, el equipo revisa lo que hizo y escucha al cliente. Si hay que mejorar algo, se hace en la siguiente iteración. Así el software se va perfeccionando poco a poco.

Mejora continua (Refactorización)

El código del programa se revisa y mejora todo el tiempo, sin cambiar lo que hace, pero haciéndolo más limpio, claro y eficiente.



Importancia de XP

XP es importante porque permite hacer software de calidad, rápido y con menos errores, especialmente cuando hay cambios frecuentes o cuando no se tiene todo completamente definido desde el inicio. También ayuda a que el equipo trabaje de manera unida, organizada y con buena comunicación.

Utilidad de XP

Proyectos donde el cliente cambia de opinión o necesita resultados en poco tiempo.

Equipos pequeños que trabajan en contacto directo con el usuario final.

Aplicaciones móviles, videojuegos o software web que se actualiza seguido.

Entornos donde se valora mucho la calidad del producto y el trabajo en equipo.

¿Qué es la Metodología ICONIX?

El enfoque ICONIX, creado por Doug Rosenberg, es un método de construcción de software basado en objetos que utiliza un sistema basado en casos de uso y una serie reducida de diagramas de modelado UML. Se sitúa entre métodos más pesados como RUP y otros más ligeros más que la XP. ICONIX proporciona una guía práctica para navegar de requisitos al código de forma estructurada, evitando sobrecargas documentales.

Características Principales

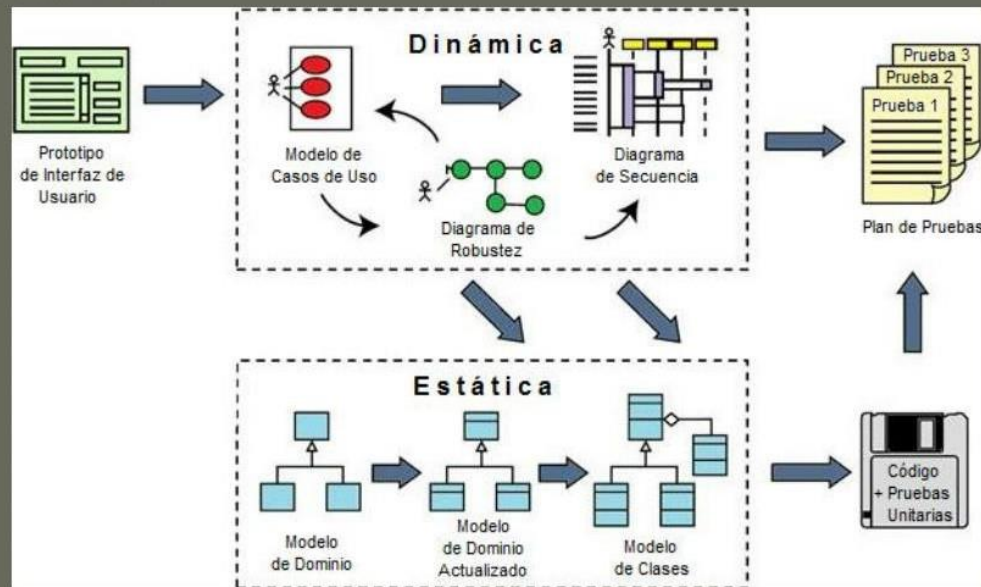
Centrada en Casos de Uso: La planificación del sistema parte de identificar interacciones entre los usuarios y el sistema a través de casos de uso bien detallados.

Pocos Diagramas UML: Solo se emplean cuatro tipos: diagrama de casos de uso, de dominio, de robustez y de secuencia.

Análisis de Robustez: Paso clave que conecta el análisis con el diseño, validando los casos de uso mediante diagramas que identifican controladores, entidades y límites.

Proceso Iterativo y Ligero: Aunque no es una metodología ágil per se, puede integrarse fácilmente con prácticas ágiles.

- **ICONIX** se divide en dos flujos de trabajo: Estático y Dinámico.



Etapas del Proceso ICONIX

Revisión de Requisitos

Se identifican ejemplos de uso, se crea por dominio y se pueden prototipar en pantallas.

Diseño Preliminar

Esta etapa ayuda a detectar errores antes de pasar al diseño detallado

Diseño Detallado

Se diseñan los esquemas de clases y los esquemas de flujo para cada escenario funcional, facilitando así la organización del código de forma sistemática y en concordancia con los requisitos

Implementación y Pruebas

El diseño se traduce en código y se implementan pruebas unitarias. La documentación producida facilita las tareas de mantenimiento y validación posteriores

Ventajas del ICONIX

Reducción de errores tempranos gracias al análisis de robustez

Claridad en la transición análisis-diseño, evitando ambigüedades

Alineación con buenas prácticas UML, sin saturar el proceso

Documentación útil para mantenimiento y futuras ampliaciones

Conclusión

Las metodologías de desarrollo de software son herramientas esenciales para que los equipos trabajen mejor, entreguen productos de calidad y se adapten a las necesidades de los clientes.

Las metodologías ágiles son perfectas para proyectos rápidos y cambiantes.

Los métodos tradicionales son ideales para proyectos con planificación detallada y estable.

Las metodologías híbridas combinan ambos mundos, siendo útiles en entornos más complejos o variados.

La metodología XP o Programación Extrema es una forma ágil, rápida y efectiva de desarrollar software. A través de prácticas como la programación en pareja, las pruebas constantes y la mejora continua, XP permite que los equipos trabajen con confianza, entreguen resultados frecuentes y mantengan una buena relación con los clientes.

La metodología ICONIX ofrece un enfoque equilibrado para el desarrollo de software, combinando la claridad y estructura de metodologías tradicionales con la agilidad y flexibilidad requeridas en entornos modernos

Elegir la metodología correcta ayuda a que el software se haga bien, a tiempo y cumpla lo que el cliente necesita.

REFERENCIAS

Maida, E. G., & Pacienza, J. (2015). *Metodologías de desarrollo de software* [Tesis de Licenciatura, Universidad Católica Argentina]. Universidad Católica Argentina.

Molina, B., Vite, H., & Dávila, J. (2018). Metodologías ágiles frente a las tradicionales en el proceso de desarrollo de software. *MQRInvestigar*, 8(1), 5052–5074. <https://doi.org/10.56048/MQR20225.8.1.2024.5052-5074>

ICONIX Process. (s.f.). Disponible en: <https://iconixprocess.wordpress.com>

Rosenberg, D., & Stephens, M. (2007). *Use Case Driven Object Modeling with UML: Theory and Practice*. Apress.

ESIC Business & Marketing School. (2024). ¿Qué es la programación extrema? Características. Recuperado de <https://www.esic.edu/rethink/business/que-es-la-programacion-extrema-caracteristicas-c>